QALight

Frontend-розробка

Заняття 12. Основи JavaScript

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Концепція семантичної розмітки

Семантичні елементи (section, nav, footer, header, article, main, aside)

Розробка унікального стилю для власного сайта.

Основи JavaScript

змінні, типи, оператори та умовні конструкції

створення арифметичного калькулятора

JavaScript

Теорія: JavaScript

JavaScript (JS) - динамічна, об'єктно-орієнтована прототипна мова програмування. Найчастіше використовується для створення сценаріїв веб сторінок, що надає можливість на боці клієнта взаємодіяти з користувачем, керувати браузером, асинхронно обмінюватися даними з сервером, змінювати структуру та зовнішній вигляд веб сторінки.

JavaScript класифікують як прототипну, скриптову мову програмування з динамічною типізацією. JS частково підтримує інші парадигми програмування (імперативну та функціональну) і деякі архітектурні властивості, зокрема: динамічна та слабка типізація, автоматичне керування пам'яттю, прототипне наслідування, функції як об'єкти першого класу.

JavaScript існує з 1995 року під назаою **LiveScript**. Авторство належить Брендану Айх (Brendan Eich). З кінця 1996 року мова JavaScript стандартизована європейською асоціацією **Ecma International**, що займається стандартизаціями інформаційних та комунікаційних систем (**ECMA** - **European Computer Manufacturers Association**). Актуальною на даний момент є версія ES2020, що вийшла у червні 2020 року.

Стандарт ECMA-262

Теорія: JavaScript

Стандартизована версія JavaScript - **ECMAScript**, працює однаково у всіх додатках, що підтримують стандарт ECMA-262. ECMA-262 визначає такі частини мови:

- синтаксис;
- типи;
- інструкції;
- ключові слова;
- зарезервовані слова;
- оператори ;
- об’єкти.

Повна реалізація JavaScript у браузері складається з трьох частин:

- ядро (ECMAScript);
- об’єктна модель документа (Document Object Model, DOM) ;
- об’єктна модель браузера (Browser Object Model, BOM).

Елемент script

Теорія: JavaScript

Для вставки JavaScript-кода у HTML-сторінку використовується елемент <script>. Він має шість атрибутів: **async,** charset, **defer,** language, **src,** type.

Атрибут defer (необов'язковий). Вказує, що виконання сценарію можна відкласти, поки не буде повністю завершено синтаксичний аналіз та візуалізація контенту документа. **Дійсний лише для зовнішніх сценаріїв.**

```
<script src=./js/index.js defer></script>
```

Атрибут async застосовується **лише для зовнішніх сценаріїв** і вимагає від браузера **невідкладно завантажити та виконати сценарій**:

```
<script src=./js/index.js async></script>
```

Розміщення script

Теорія: JavaScript

JavaScript-код можна **вбудовувати** безпосередньо у сторінку або **підключати** сценарій із **зовнішнього файлу**. Приклад **вбудованого** JavaScript-код:

```
<script> alert( "Hi!" ) ; </script>
```

JavaScript код - однопотоковий і **обробляється зверху вниз**. Контент сторінки не завантажується та не відображається, поки не буде виконано код в елементі script.

Для включення JavaScript-коду із зовні, використовують атрибут **src**. Його значенням має бути **URL-адреса файлу зі сценарієм**. Файл повинен **містити лише JavaScript-код.**

Зовнішнім JavaScript-файлам призначають розширення **.js**, проте це не обов'язково, оскільки браузери не перевіряють розширення підключених файлів, але сервери часто використовують розширення файлу для визначення правильного MIME-типу.

Основи синтаксису

Теорія: JavaScript

JavaScript оперує зі змінними, що називаються **identifiers.** Імена змінних **чутливі до регістру. У якості кодування використовують Unicode.** Ідентифікатори разом з операторами створюють інструкції (**statements).**

Інструкції розділяються **крапкою з комою (;).** За допомогою фігурних дужок **{ }** кілька інструкцій можна об’єднати у блоки кода.

JavaScript підтримує однорядкові та блокові коментарі. Однорядковий коментар починається з двох скісних рисок (//). Однорядковий коментар перетворює весь текст, наступний за // у тому самому рядку, на коментар:

```
// this is a single-line comment
```

Блоковий коментар починається з скісної риски та зірочки /* і закінчується протилежним */, як у наступному прикладі:

```
/* This is a block comment
that can span multiple lines */
```

Послідовності елементів

Теорія: JavaScript

Текст скрипта JavaScript сканується зліва направо та конвертується у **послідовність вихідних елементів** (токенів **- tokens). Токен - це управляючий символ**, символ кінця рядка, коментар або символ пробілу.

Пробіл (**space**), табуляція (**tab**) та новий рядок (**newline**) називаються пробільними символами (**whitespace**). Не можна змішувати пробіл і знак табуляції.

Лексема (token) - мінімальна одиниця мови, що має самостійний смисл

- імена (**ідентифікатори**);
- ключові слова;
- символи операцій;
- розділителі;
- літерали (константи).

Ідентифікатори

Теорія: JavaScript

Ідентифікатор (**identifiers)** – це ім’я, яке ви обираєте для змінних, параметрів, функцій, класів тощо.

Ім’я ідентифікатора починається з літери (a-z або A-Z), підкреслення (_) або знака долара ($), після якого йде послідовність символів, включаючи (a-z, A-Z), цифри (0-9), підкреслення (_) і знаки долара ($). Літера не обмежується символом ASCII і може включати розширений ASCII або Unicode, хоча це **не рекомендується**.

Ідентифікатори **чутливі до регістру**. Наприклад, **_Number** відрізняється від **_number**.

JavaScript визначає список **зарезервованих ключових слів**, які мають певне використання. Не можна використовувати зарезервовані ключові слова як ідентифікатори.

Зарезервовані слова JavaScript, визначені в ECMA-262: break, case, catch, continue, debugger, default, else, export, extends, function, if, import, new, return, super, throw, try, null, void, while, with, class, delete, finally, in, switch, typeof, yield, const, do, for, instanceof, this, var

Окрім зарезервованих ключових слів, ECMA-252 також визначає список майбутніх зарезервованих слів, які не можна використовувати як ідентифікатори чи назви властивостей: enum, implements, let, protected, private, public, await, interface, package, implements, public

Strict mode

Теорія: JavaScript

Режим за замовчуванням у JavaScript називається **недбалим (sloppy)**. Строгий режим JavaScript є способом підключення до обмеженого варіанту JavaScript, відмовляючись від **sloppy режиму**. Строгий режим вносить кілька змін до звичайної семантики JavaScript:

- Усуває деякі помилки JavaScript, шляхом обробки виключень.
- Виправляє помилки, через які важко виконувати оптимізацію: іноді код строгого режиму може виконуватися швидше, ніж ідентичний код **недбалого режиму**.
- Забороняє певний синтаксис, який буде визначено в майбутніх версіях ECMAScript.

Строгий режим застосовується до сценаріїв або до окремих функцій. Щоб викликати строгий режим, вставте директиву **"use strict"**; (або '**use strict**';) перед будь-якими іншими операторами.

```
"use strict"
const v = "Hi! I'm a strict mode script!";
```

Якщо встановлено строгий режим, **відмінити його неможливо**.

Глобальні змінні

Теорія: JavaScript

Глобальні змінні - це **властивості глобального об'єкта**. На веб-сторінках глобальним об'єктом є **window**. Можна встановити глобальні змінні за допомогою синтаксису **window.variable:**

```
window.foo = 'Hello bar!';
```

Функції

Теорія: JavaScript

Функції - це спосіб упаковки функціональних можливостей, які ви бажаєте використовувати повторно. Тіло коду можна визначити як функцію, яка виконується, коли ви викликаєте назву функції у своєму коді. Функції є хорошою альтернативою багаторазовому написанню одного й того ж коду. Багато функцій вбудовано в браузер.

Функції часто приймають аргументи. Аргументи вказуються в дужках, розділяючись комами, якщо аргументів більше одного. Наприклад, функція alert() створює спливаюче вікно у вікні браузера. **Виконання функції називають викликом**: alert("hello!");

Ви також можете визначити власні функції. У наступному прикладі ми створюємо просту функцію, яка приймає два числа як аргументи та множить їх:

```
function multiply(num1, num2) {
  let result = num1 * num2;
  return result;
}
```

Налагодження JavaScript

Теорія: JavaScript

Панель Debugger (**Sources** у Chrome) - це середовище розробки кода, де можна отримати доступ до всіх підключених до сторінки файлів.

JavaScript має функцію **console.log,** що виводить результат на консоль браузера.

```
var x = 30;
console.log("the value of x is", x);
// → the value of x is 30
```

Діалоги

Теорія: JavaScript

Alert є функцією, що показує вікно з повідомленням:

```
alert("Good morning Cats!");
```

Параметри, що передають функціям, називаються **аргументами функцій**. Функція **alert вимагає один аргумент**. alert(window.foo);

Можна викликати **вікно з питанням і кнопками OK/Cancel** за допомогою функції **confirm**. Вона повертає **булеве** значення – **true**, якщо натиснути OK, і **false**, якщо натиснути Cancel.

```
confirm("Well, let's go?");
```

Функцію **prompt** можна використовувати, щоб задать питання. Перший аргумент – питання, другий – текст - початок відповіді користувача.

```
prompt("Dude, tell me everything you know about programming", "...");
```

Зовнішні скрипти

Теорія: JavaScript

Модуль - це файл. Один скрипт – це один модуль. Необхідно явно сказати браузеру, що скрипт є модулем, з допомогою атрибута **<script type="module">**. Модулі мають ряд особливостей:

- Відкладене (deferred) виконання за замовчуванням.
- Атрибут async працює у вбудованих скриптах.
- Для завантаження модулів з іншого джерела повинні бути встановлені заголовки CORS.
- Зовнішні скрипти, що дублюються, ігноруються.
- Модулі мають свою область видимості.
- Директива use strict завжди ввімкнена в модулі.
- Код у модулях виконується лише один раз.

Модулі працюють тільки через HTTP(s), не локально

Якщо ви спробуєте відкрити веб-сторінку локально через file:// протокол, то ви побачите, що директиви import/export не працюють. Використовуйте локальний веб-сервер, такий як static-server або скористайтеся можливістю “живого сервера” вашого редактора, наприклад, VS Code Live Server Extension для тестування модулів.

Ключове слово var

Теорія: JavaScript

Змінні JavaScript можна визначати за допомогою **ключового слова var**.

var створює змінні, що потрапляють до глобального об'єкту, або до області видимості функцій.

```
var thing = 'first';
var a = 2;
```

var не звертає уваги на блоки кода: `{ var a = 3; }`

Крім того, можна звертатись до змінної, що об'явлена ключовим словом var до її визначення.

```
console.log("the value of cat is ", cat);
var cat = 'Red Cat';
```

Це явище називають **підняття змінних (hoisting)**.

Ключові слова let і const

Теорія: JavaScript

Змінні, що визначені за допомогою ключових слів **let і const**, мають **блочну область видимості.** Використання **let та const не допускає звернення до змінної поки вона не визначена**.

```
    let a   = 10;
    console.log("the value of a is ", a);
  { let a = 3;   console.log("the value of a inside block is ", a); }
   console.log("the value of a outside block is ", a);
   console.log("the value of cat is ", cat); //  ReferenceError: can't access lexical declaration `cat' before initialization
```

Значення змінної, що визначена за допомогою ключового слова **const**, не можна змінювати.

```
const NAME = 'Maybe';
const NAME = 'Red Cat';
```

Оператори

Теорія: JavaScript

JavaScript підтримує **бінарні** та **унарні** оператори, а також спеціальний **тернарний оператор** - умовний оператор.

Бінарна операція використовує два операнди:

3+4 або x*y.

Унарна операція використовує один операнд:

x++

або

++x

Оператори привласнення

Теорія: JavaScript

Основним оператором привласнення є **=.**

Вираз x = y означає, що x привласнюється значення y.

```
// declaring variables
   let a   = 10;
   let b   = 3;
   let c   = 15;
   let d = '5';
   let e   = 'My name';
   let f = 'Maybe';
```

Арифметичні оператори

Теорія: JavaScript

Арифметичні оператори використовують у якості операндів числа та повертають числовий результат. Стандартні арифметичні оператори

- Додавання (+)
- Віднімання (-)
- Множення (×)
- Ділення (/)
- Залишок від ділення **(%)**

Ділення на нуль повертає нескінченність (Infinity).

```
console.log(1/2); /* 0.5 */
```

Залишок від ділення (%) двух операндів.

```
let b   = 3;
let c   = 15;
console.log(c % b);
```

Оператор групування

Теорія: JavaScript

```
// Оператор групування ( ) контролює порядок виконання у виразах.
var a = 1;
var b = 2;
var c = 3;
console.log(a + b * c);     // 7 звичайний порядок
console.log(a + (b * c));   // 7
console.log((a + b) * c);   // 9
console.log(a * c + b * c); // 9
```

Унарні оператори

Теорія: JavaScript

Інкремент (++) - додає одиницю до операнду. У префіксній формі (++x) повертає значення операнда з добавленою до нього одиницею. У суфіксній формі (x++) повертає величину операнда перед додаванням до нього одиниці. Якщо x = 3, тоді ++x встановить і повертає значення 4, проте x++ поверне 3 і лише потім встановить значення x = 4.

Декремент (--) - віднімає одиницю від свого операнда. Якщо x = 3, тоді --x встановить і повертає значення 2, x-- поверне 3 і лише потім встановить значення x = 2.

Унарний мінус - повертає від'ємне значення операнда.

Унарний плюс (+) - спробує конвертувати операнд у число. +"3" поверне 3.

Ступінь числа (**) - 2 ** 3 = 8.

Типи даних

Теорія: JavaScript

В ECMAScript існує **6 простих типів даних - примітивних** типів (**primitive types**):

1. невизначений (undefined) ,
2. нульовий (null),
3. логічний (boolean) ,
4. числовий (number)
5. рядковий (string).
6. Symbol для створення унікальних ідентифікаторів.

Складний тип даних (**object**) - це не впорядкований список пар імен та значень.

Визначити власні типи даних у ECMAScript неможливо. Усі значення представляють за допомогою одного з цих типів.

Тип Null і Undefined

Теорія: JavaScript

Спеціальне значення **null** - це окремий тип, що містить **лише значення null**: var car=null;

В JavaScript **null не є** посиланням на неіснуючий об’єкт чи нульовим вказівником. Це **спеціальне значення**, що еквівалентне нічого, пусто або значення невідоме. **Логічно null** - це вказівник на пустий об’єкт, тому оператор typeof повертає для нього **object**: alert(typeof car); // "object "

Невизначений тип (**undefined**) містить спеціальне значення undefined.

```
var messageUndifined;
console.log(messageUndifined == undefined);
```

Значення undefined є **похідним від null**, в ECMA-262 вони визначені як **нестрого рівні**:

```
alert ( null == undefined ) ; // true
```

Булевий (логічний) тип

Теорія: JavaScript

Логічний тип (**boolean**) має лише два літеральних значень: **true** и **false**. Вони відрізняються від числових значень: true не дорівнює 1, а false не дорівнює 0.

```
var found = true;
var lost = false;
```

Літерали true і false чутливі до регістру, **True і False не є логічними значеннями**.

Тип number

Теорія: JavaScript

Цілі числа представляються за допомогою двійкових, вісімкових, десяткових чи шістнадцяткових літералів. У вісімковому літералі першою цифрою є нуль(0), далі прямує послідовність вісімкових цифр (від 0 до 7). У строгому режимі після нуля має стояти символ o:

```
var octaNum = 0o70 ; // 56
var octalNum3 0o8; // неприпустиме значення
```

Для визначення **значень з плаваючою крапкою**, необхідно ввести десяткову крапку та хочаб одну цифру після неї. Нуль перед десятковою крапкою не обов'язковий:

```
var floatNum2=0.1;
```

Дуже великі та маленькі числа з плаваючою крапкою можна представляти у **експоненціальному форматі** (**e-notation**):

```
var floatNum=3.125e7 // 31250000
```

Помилки округлення

Теорія: JavaScript

// У JavaScript тип «число» не може безпечно представляти цілі значення, більші за (253-1) (це 9007199254740991) або менші за -(253-1) для негативних.
Значення з плаваючою крапкою має точність до **17-го десяткового розряду**, вони менш точні, ніж цілі числа. Наприклад, **0.1 + 0.2 дорівнює 0.30000000000000004** замість 0.3.

```
if ( a + b == 0.3 ) {
 alert ( "You got 0.3 . " );
}
```

Цей код працює правильно, якщо додавати 0.05 і 0.25 або 0.15 і 0.15, проте для чисел 0.1 і 0.2 результат буде дещо більший.

Помилки округлення - це побічний ефект арифметичних операцій над числами з плаваючою крапкою формата **IEEE-754**, а не особливість ECMAScript.

```
Тип BigInt для представлення цілих чисел довільної довжини.
// Значення BigInt створюється шляхом додавання n до кінця цілого числа: "n"
const bigInt = 1234567890123456789012345678901234567890n;
```

Рядки

Теорія: JavaScript

Рядки записують у лапках: **‘Everyone's stealing my code Hello world’**
JavaScript підтримує три типи лапок.

- Подвійні: "**Hello world**".
- Одинарні: '**Hello world**'.
- Зворотні: `**Hello world**`.

Подвійні або одинарні лапки є простими, між ними немає різниці у JavaScript. Зворотні лапки мають розширену функціональність, що дозволяє вбудовувати виразу у рядок:

```
let name = "Cats";
console.log(`Hello, ${name}!`); // Hellj, Cats!
```

Оператор конкатенації (+) об’єднує два рядка, повертаючи новий рядок.

```
console.log("my " + "string");  // "my string".
```

Скорочений оператор += також може використовуватись для об’єднання рядків.

```
var  mystring = "alpha";
mystring += "bet"; // "alphabet"
```

Тип Symbol

Теорія: JavaScript

Тип Symbol є простим типом даних разом. На відміну від рядків, чисел або логічних значень, для символа не існує літералу. Його потрібно створити за допомогою функції Symbol().
Кожне значення, яке повертає Symbol(), є унікальним, тому два символи ніколи не можуть мати однакові значення.

Символи - це значення, які програми можуть створювати та використовувати як ключі властивостей без ризику зіткнення імен.

var mySymbol = Symbol();

Виклик Symbol() створює новий символ, значення, яке не дорівнює жодному іншому значенню. Так само, як рядок або число, можна використовувати символ як ключ властивості. Оскільки воно не дорівнює жодному рядку, ця властивість символьного ключа гарантовано не конфліктуватиме з будь-якою іншою властивістю.

Об’єкт JavaScript

Теорія: JavaScript

Об’єкт javascript - це невпорядкована колекція пар **ключ-значення**. Кожна пара ключ-значення називається **властивістю**. **Ключем** властивості може бути рядок. А **значенням** властивості може бути будь-яке значення, наприклад, рядок, число, масив і навіть функція. Якщо об’єкт має кілька властивостей, використовується кома (,), щоб розділити їх.

Значенням властивості може бути функція, яку називають **методом** об’єкта.

Властивість об’єкта - це змінна, що закріплена за об’єктом, і є звичайною змінною JavaScript. Щоб створити об’єкт із властивостями, використовується ключ:значення у фігурних дужках. Наприклад, наступне створює новий об’єкт person:

```
let person = {
        firstName: 'John',
        lastName: 'Doe'
};
```

Отримати доступ до властивості об’єкта можна за допомогою дот-нотації: **object.property**
Ім’я об’єкта та імена властивостей **чутливі до регістру**.

Функції JavaScript

Теорія: JavaScript

Функція - це засіб для структурування великих програм, зменшення повторень, та ізолювання підпрограм одну від іншої. Функція містить **блок кода**, який можна викликати у будь-якій частині програми. Код функції виконується лише після явного виклику функції.

Функція JavaScript - **це об’єкт.** Функцію можна передавати як аргумент та повертати у якості результату при виклику інших функцій (**функцій вищого порядку**). Функцію можна створювати анонімною та привласнювати у якості значення властивості об’єкту.

Функції, згідно ECMAScript, вважаються об’єктами, а не окремим типом даних.
Однак функції мають деякі спеціальні властивості, тому коли ви використовуєте оператор typeof для функції, ви отримуєте як результат функцію.

```
console.log(typeof(function(){}));
```

Оператор typeof

Теорія: JavaScript

Для визначення типу змінної використовують оператор typeof, що повертає:

- undefined, якщо значення не визначено;
- boolean, якщо значення має логічний тип;
- string, якщо значення є рядком;
- number, якщо значення є числом;
- object, якщо значення є об’єктом або null;
- function, якщо значення є функцією.

```
typeof undefined // "undefined"
typeof 0 // "number"
typeof 10n // "bigint"
typeof true // "boolean"
typeof "foo" // "string"
```

Оператор порівняння

Теорія: JavaScript

Оператор порівняння **повертає логічне значення**, базуючись на рівності чи нерівності значень операндів. Операнди можуть бути числами, рядками, логічними величинами або об’єктами.

Рядки порівнюються на основі стандартного лексикографічного порядку, використовуючи **Unicode-значення**. Якщо операнди мають різний тип, JavaScript намагається перетворити відповідний тип у підходящий для порівняння.

- **Дорівнює (==)**: Повертає true, якщо операнди рівні. 3 == '3'
- **Не дорівнює (!=)**: Повертає true, якщо операнди не рівні. var1 != 4 var2 != "3"
- **Більше (>)**: Повертає true, якщо операнд зліва більше операнда справа.
- **Більше або дорівнює (>=)**: Повертає true, якщо операнд зліва більше або дорівнює операнду справа.
- **Менше (<)**: Повертає true, якщо операнд зліва менше операнда справа.
- **Менше або дорівнює (<=)**: Повертає true, якщо операнд зліва менше або дорівнює операнду справа.
- **Строго дорівнює (===)**: Повертає true, якщо операнди рівні та мають однаковий тип.
- **Строго не дорівнює(!==)**: Повертає true, якщо операнди не рівні та мають різний тип

Порівняння з використанням операторів **=== і !== - строге порівняння,** не виконують перетворення типів операндів перед їх порівнянням.

Значення NaN

Теорія: JavaScript

Спеціальне значення NaN (**Not a Number**), вказує, що операція, яка б мала повернути число, **цього не зробила, проте й не сгенерувала помилку**. Наприклад **ділення на 0 повертає значення NaN**.

Будь-яка операція з NaN завжди **повертає NaN**. **NaN не дорівнює ніякому значенню, в тому числі й NaN**: alert(NaN==NaN) ; //false

Функція isNaN() приймає аргумент будь-якого типу та визначає, чи є він **не числом**.

```
console.log( isNaN(NaN));   // true
console.log( isNaN(10)) ;   // false
console.log( isNaN("10"));  // false
console.log( isNaN("Ыlue")); // true
console.log( isNaN(true));  // false
```

Перетворення рядків

Теорія: JavaScript

У більшості випадків оператори та функції **автоматично** перетворюють надані їм значення в правильний тип. Наприклад, **alert(value)** автоматично перетворює будь-яке значення на рядок, щоб показати його. Математичні операції перетворюють значення на числа.

Бувають також випадки, коли потрібно явно перетворити значення в очікуваний тип. Перетворення рядків відбувається, коли нам потрібна рядкова форма значення.

Також можна викликати функцію **String(value)**, щоб перетворити значення на рядок:

```
let value = true;
alert(typeof value); // boolean
value = String(value); // now value is a string "true"
alert(typeof value); // string
```

Конверсія рядка здебільшого очевидна. false стає "false", null стає "null" тощо.

Перетворення чисел

Теорія: JavaScript

Існує 3 функції перетворення значень у числа: Number(), parseInt() і parseFloat(). Функции parseInt() і parseFloat() перетворюють рядки у числа. Функція **Number**() працює за такими правилами.

- Логічні значення true і false перетворюються в 1 і 0 відповідно.
- Значення null перетворюється в 0.
- Значення undefined перетворюється в NaN.
- Рядок цифр зі знаком або без знака перетворюється в десяткове число.
- Якщо рядок містить значення з плаваючою крапкою, він перетворюється в **Float-**число.
- Пустий рядок перетворюється в 0. В інших випадках пядок перетворюється в NaN.

```
// Унарний оператор «плюс» працює так, як функція Number().
  (+"1.1") + (+"1.1") // 2.2
```

Функція **parseFloat(arg)** аналізує аргумент і повертає число з плаваючою комою, або NaN, коли перший непробільний символ не можна перетворити на число. Якщо аргумент **arg** не є рядком, він перетворюється на такий за допомогою абстрактної операції **ToString**. Пробіли на початку аргументу ігноруються. JavaScript **не розрізняє числа з плаваючою комою та цілі числа**. Функції **parseInt() і parseFloat()** відрізняються лише поведінкою синтаксичного аналізу.

Наприклад, parseInt("42") і parseFloat("42") повернуть те саме значення: число 42.

Округлення чисел

Теорія: JavaScript

Метод toFixed(digits) форматує число за допомогою нотації з фіксованою комою. **digits** - необов'язковий параметр - кількість цифр після коми; це може бути значення від 0 до 20 включно. Якщо цей аргумент опущено, він розглядається як 0.

Можуть виникати такі винятки

- RangeError - Якщо цифри занадто малі або занадто великі. Значення від 0 до 100 включно не викликають RangeError. Реалізації можуть підтримувати більші та менші значення за вибором.
- TypeError - Якщо цей метод викликається для об’єкта, який не є числом.

toFixed() повертає рядкове представлення числа, яке не використовує експоненціальний запис і містить точну кількість цифр після коми. За потреби число округлюється, а дробова частина доповнюється нулями, щоб вона мала задану довжину. Якщо абсолютне значення більше або дорівнює 1e+21, цей метод викликає .toString() і повертає рядок у експоненціальній нотації.

```
let numObj = 12345.6789
numObj.toFixed()       // Returns '12346': rounding, no fractional part
numObj.toFixed(1)      // Returns '12345.7': it rounds up
numObj.toFixed(6)      // Returns '12345.678900': additional zeros
(1.23e+20).toFixed(2)  // Returns '123000000000000000000.00'
```

Логічні оператори

Теорія: JavaScript

Логічні оператори (**&&**) та (**||**) повертатимуть логічне значення **true** або **false** залежно від оцінки.
Синтаксис оператора &&: **операнд 1 && операнд 2 && ... && операнд N**

```
true && true   // => true
true && false  // => false
x > 12 && y < 25 // True
x < 2 && y >12 // False
```

Узагальнений синтаксис оператора ||: **операнд1 || операнд2 || ... || операндN**

```
true || true   // => true
true || false  // => true
x > 12 || y > 25 // True
x < 2 || y < 12 // False
```

Оператор bang

Теорія: JavaScript

У найпростішій формі оператор **bang** просто заперечує все, що ви передаєте після нього.

```
!true // false
!false // true
```

Проте оператор стає дещо складнішим, коли ви маєте справу зі значеннями, які ще не є логічними. Щоб правильно працювати, оператор bang повинен спочатку перетворити будь-яке значення, для якого він використовується, у логічне значення, що іноді має непередбачені наслідки.

```
!null // true
!undefined // true
!{ name: "Kyle" } // false
null == null // true
undefined == null // true
{ name: "Kyle" } == null // false
```

Умовні переходи

Теорія: JavaScript

Умовні оператори можуть контролювати потік виконання коду. Роблячи це, вони діють як **тригери** для блоків коду, які потрібно виконати, коли виконуються певні умови. Умовні оператори використовують оператори порівняння та логічні оператори для оцінки умови та повернення істинного чи хибного значення на основі оцінки. Умовний оператор записують за допомогою **ключового слова if**.

```
if(//conditional){
    /**executed code if conditional is true**/
}
//continue with code
```

Виконати якийсь код тільки тоді, коли виконується якась умова.

```
var theNumber = prompt("Number ", "");
if (!isNaN(theNumber)){
    console.log("theNumber * theNumber = " +  theNumber * theNumber);
}
```

Ключове слово else

Теорія: JavaScript

Ключове слово **else** використовується разом з if.

```
var num = Number(prompt("Вибери число", ""));

if (!isNaN(num)) {
      alert("Квадрат числа = " + num * num;
} else {
       alert("ЦЕ не схоже на число");
}
```

Можна використовувати **кілька пар if/else**

```
if (num < 10) {  alert("No");}
else if (num < 100) {alert("Ok");}
else { alert("Nop"); }
```

Оператор switch

Теорія: JavaScript

Інструкція switch - це спосіб приймати рішення у коді на основі різних умов. Це більш організована та стисла альтернатива використанню кількох операторів if-else. Інструкція switch обчислює заданий вираз, який може бути **змінною** або **значенням**, і порівнює його з кількома можливими випадками. Якщо значення виразу відповідає одному з випадків, виконується відповідний блок коду. Якщо відповідності не знайдено, необов'язковий випадок за замовчуванням може бути виконаний як запасний, тобто він запускається, коли жоден із інших випадків не застосовується. Наприклад, ось простий оператор switch, який перевіряє значення змінної під назвою day:

```
switch (day) {
 case "Monday":
   console.log("Start of the work week!");
   break;
 case "Friday":
   console.log("End of the work week!");
   break;
 default:
   console.log("A regular day");
}
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua